



Faculty of Engineering - University of Ruhuna

Software Requirement Specification

<Project Name>

Version 1.0

7/9/24

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

VERSION AND APPROVALS

VERSION HISTORY			
<u>Version #</u>	<u>Date</u>	<u>Revised By</u>	<u>Reason for change</u>

This document has been approved as the official Software Requirements Document for <project name>, and accurately reflects the current understanding of business requirements. Following approval of this document, requirement changes will be governed by the project's change management process, including impact analysis, appropriate reviews and approvals.

DOCUMENT APPROVALS			
<u>Approver Name</u>	<u>Project Role</u>	<u>Signature/Electronic Approval</u>	<u>Date</u>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

PROJECT DETAILS

Project Name	Enter Project Name
Project Type	<i>(e.g. New Initiative or Phase II)</i>
Project Start Date	
Project End Date	
Project Sponsor	
Primary Driver	<i>(e.g. Mandatory or Efficiency)</i>
Secondary Driver	
Division	
Project Manager/Dept	

OVERVIEW

This document defines the high level requirements [insert project name]. It will be used as the basis for the following activities:

- Creating solution designs
- Developing test plans, test scripts, and test cases
- Determining project completion
- Assessing project success

DOCUMENT RESOURCES

Name	Business Unit	Role
<Identify all stakeholders and resources involved in gathering requirements>		

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Name	Business Unit	Role

GLOSSARY OF TERMS

Term/Acronym	Definition
<Identify any terms and acronyms used within this document>	

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Contents

VERSION AND APPROVALS	1
PROJECT DETAILS.....	2
OVERVIEW.....	2
DOCUMENT RESOURCES	2
GLOSSARY OF TERMS.....	3
INTRODUCTION.....	7
1.1 Purpose	7
1.2 Document Conventions	7
1.3 Intended Audience and Reading Suggestions.....	7
1.4 Product Scope	7
1.5 References.....	7
OVERALL DESCRIPTION	8
2.1 Product Perspective	8
2.1.1 System Interfaces	8
2.1.2 Interfaces.....	8
2.1.3 Hardware Interfaces.....	8
2.1.4 Software Interfaces	9
2.1.5 Communications Interfaces.....	10
2.1.6 Memory Constraints.....	10
2.1.7 Operations.....	10
2.1.8 Site Adaptation Requirements	10
2.2 Product Functions	11
2.3 User Classes and Characteristics	11
2.4 Operating Environment.....	11
2.5 Design and Implementation Constraints	12
2.6 Stakeholders.....	12
2.7 User Documentation	12
2.8 Assumptions and Dependencies	13
2.9 Apportioning of Requirements.....	13
SYSTEM FEATURES / FUNCTIONS.....	14
3.1 System Feature 1.....	14
3.1.1 Description and Priority.....	14

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

3.1.2	Stimulus/Response Sequences	14
3.1.3	Functional Requirements	14
3.1.4	Use case diagram	16
3.1.5	Sequence Diagram.....	22
3.1.6	System Mock-up Screens.....	22
3.2	System Feature 2 (and so on)	23
3.3	Database Design	24
	OTHER NONFUNCTIONAL REQUIREMENTS	25
4.1	Performance Requirements	25
4.2	Safety Requirements	25
4.3	Security Requirements	25
4.4	Software Quality Attributes.....	25
4.5	Business Rules	26
	OTHER REQUIREMENTS.....	27
5.1	Online User Documentation and Help System Requirements.....	27
5.2	Purchased Components	27
5.3	Interfaces.....	27
5.3.1	User Interfaces	27
5.3.2	Hardware Interfaces.....	27
5.3.3	Software Interfaces	27
5.3.4	Communications Interfaces.....	28
5.4	Licensing Requirements	28
5.5	Legal, Copyright and Other Notices.....	28
5.6	Applicable Standards.....	28
	SYSTEM ARCHITECTURE.....	29
6.1	Functional Architecture	29
6.2	Logical Architecture.....	29
6.3	Component Architecture.....	29
6.4	Physical Architecture	30
	APPENDIX A: GLOSSARY	31
	APPENDIX B: ANALYSIS MODELS.....	32
	APPENDIX C: TO BE DETERMINED LIST.....	33

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

INTRODUCTION

1.1 Purpose

<Identify the product whose software requirements are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this SRS, particularly if this SRS describes only part of the system or a single subsystem.>

1.2 Document Conventions

<Describe any standards or typographical conventions that were followed when writing this SRS, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.>

1.3 Intended Audience and Reading Suggestions

<Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this SRS contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.>

1.4 Product Scope

<Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here.>

1.5 References

<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

OVERALL DESCRIPTION

<Describe the general factors that affect the product and its requirements. This section does not state specific requirements. Instead, it provides a background for those requirements, which are defined in section 3, and makes them easier to understand. In a sense, this section tells the requirements in plain English for the consumption of the customer>

2.1 Product Perspective

<Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two. A simple diagram that shows the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.>

2.1.1 System Interfaces

<List each system interface and identify the functionality of the software to accomplish the system requirement and the interface description to match the system. These are external systems that you have to interact with. For instance, if you are building a business application that interfaces with the existing employee payroll system, what is the API to that system that designers will need to use?>

2.1.2 Interfaces

Specify:

- (1) The logical characteristics of each interface between the software product and its users.*
- (2) All the aspects of optimizing the interface with the person who must use the system*

This is a description of how the system will interact with its users. Is there a GUI, a command line or some other type of interface? Are there special interface requirements? If you are designing for the general student population for instance, what is the impact of ADA (American with Disabilities Act) on your interface?

2.1.3 Hardware Interfaces

Specify the logical characteristics of each interface between the software product and the hardware components of the system. This includes configuration characteristics. It also covers such matters as what devices are to be supported, how they are to be supported and protocols. This is not a description of hardware requirements in the sense that "This program must run on a Mac with 64M

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

of RAM". This section is for detailing the actual hardware devices your application will interact with and control. For instance, if you are controlling X10 type home devices, what is the interface to those devices? Designers should be able to look at this and know what hardware they need to worry about in the design. Many business type applications will have no hardware interfaces. If none, just state "The system has no hardware interface requirements" If you just delete sections that are not applicable, then readers do not know if: a. this does not apply or b. you forgot to include the section in the first place.

2.1.4 Software Interfaces

Specify the use of other required software products and interfaces with other application systems. For each required software product, include:

- (1) Name
- (2) Mnemonic (Encoding)
- (3) Specification number
- (4) Version number
- (5) Source

For each interface, provide:

- (1) Discussion of the purpose of the interfacing software as related to this software product
- (2) Definition of the interface in terms of message content and format

Here we document the APIs, versions of software that we do not have to write, but that our system has to use. For instance, if your customer uses SQL Server 7 and you are required to use that, then you need to specify i.e.

Microsoft SQL Server 7. The system must use SQL Server as its database component. Communication with the DB is through ODBC connections. The system must provide SQL data table definitions to be provided to the company DBA for setup.

A key point to remember is that you do NOT want to specify software here that you think would be good to use. This is only for **customer-specified systems** that you **have** to interact with. Choosing SQL Server 7 as a DB without a customer requirement is a Design choice, not a requirement. This is a subtle but important point to writing good requirements and not over-constraining the design.

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

2.1.5 Communications Interfaces

Specify the various interfaces to communications such as local network protocols, etc. These are protocols you will need to directly interact with. If you happen to use web services transparently to your application, then do not list it here. If you are using a custom protocol to communicate between systems, then document that protocol here so designers know what to design. If it is a standard protocol, you can reference an existing document or RFC.

2.1.6 Memory Constraints

Specify any applicable characteristics and limits on primary and secondary memory. Don't just make up something here. If all the customer's machines have only 128K of RAM, then your target design has got to come in under 128K so there is an actual requirement. You could also cite market research here for shrink-wrap type applications "Focus groups have determined that our target market has between 256-512M of RAM, therefore the design footprint should not exceed 256M." If there are no memory constraints, so state.

2.1.7 Operations

Specify the normal and special operations required by the user such as:

- (1) The various modes of operations in the user organization*
- (2) Periods of interactive operations and periods of unattended operations*
- (3) Data processing support functions*
- (4) Backup and recovery operations*

(Note: This is sometimes specified as part of the User Interfaces section.) If you separate this from the UI stuff earlier, then cover business process type stuff that would impact the design. For instance, if the company brings all their systems down at midnight for data backup that might impact the design. These are all the work tasks that impact the design of an application, but which might not be located in software.

2.1.8 Site Adaptation Requirements

In this section:

- (1) Define the requirements for any data or initialization sequences that are specific to a given site, mission, or operational mode*
- (2) Specify the site or mission-related features that should be modified to adapt the software to a particular installation*

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

If any modifications to the customer's work area would be required by your system, then document that here. For instance, "A 100Kw backup generator and 10000 BTU air conditioning system must be installed at the user site prior to software installation".

This could also be software-specific like, "New data tables created for this system must be installed on the company's existing DB server and populated prior to system activation." Any equipment the customer would need to buy or any software setup that needs to be done so that your system will install and operate correctly should be documented here.

2.2 Product Functions

Provide a summary of the major functions that the software will perform. Sometimes the function summary that is necessary for this part can be taken directly from the section of the higher-level specification (if one exists) that allocates particular functions to the software product.

For clarity:

- (1) The functions should be organized in a way that makes the list of functions understandable to the customer or to anyone else reading the document for the first time.*
- (2) Textual or graphic methods can be used to show the different functions and their relationships. Such a diagram is not intended to show a design of a product but simply shows the logical relationships among variables.*

2.3 User Classes and Characteristics

<Identify the various user classes that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.>

2.4 Operating Environment

<Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

2.5 Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

Provide a general description of any other items that will limit the developer's options. These can include:

- (1) *Regulatory policies*
- (2) *Hardware limitations (for example, signal timing requirements)*
- (3) *Interface to other applications*
- (4) *Parallel operation*
- (5) *Audit functions*
- (6) *Control functions*
- (7) *Higher-order language requirements*
- (8) *Signal handshake protocols (for example, XON-XOFF, ACK-NACK)*
- (9) *Reliability requirements*
- (10) *Criticality of the application*
- (11) *Safety and security considerations*

2.6 Stakeholders

The following comprises the internal and external stakeholders whose requirements are represented by this document:

	Stakeholders
1.	
2.	
3.	

2.7 User Documentation

<List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

2.8 Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

#	Assumptions
	<i>List any assumptions the requirements are based on</i>
#	Constraints
	<i>List any constraints the requirements are based on</i>

2.9 Apportioning of Requirements.

Identify requirements that may be delayed until future versions of the system. After you look at the project plan and hours available, you may realize that you just cannot get everything done. This section divides the requirements into different sections for development and delivery. Remember to check with the customer – they should prioritize the requirements and decide what does and does not get done. This can also be useful if you are using an iterative life cycle model to specify which requirements will map to which interaction.

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

FUNCTIONALITY REQUIREMENTS

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

3.1 Functionality 1

< State the feature name in just a few words.>

3.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

3.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

3.1.3 Functional Requirements / User Stories

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Use "TBD" as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:

REQ-2:

User stories are **short, simple explanations of a software feature** written by someone who desires a new capability.

User stories articulate how a software feature will provide value to the customer. User stories describe the type of users, what they want, and why. They create a simplified description of a requirement.

US-1

US-2

"As a [particular user], I want to [perform this action] so that [I can achieve this goal]."

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

3.1.4 User Stories

Templates

Title:	Priority	Estimate:
User story	As a [type of user], I want to [perform some task] so that I can [achieve some goal]	
Acceptance criteria	Given that [some context], when [some action is carried out] then [a set of observable outcomes should occur]	

Agile User Story Example		
As a:	salesperson	Intent: This segment defines the personas (customers) and their mindset. Answers: Who are your customers What is their mindset
I want:	a customer relationship management software	Intent: This segment defines the problems and the wants of the customers Answers: What is the end goal of the customer?
So that	I can manage all my client conversations in one place	Intent: This segment defines the aftereffect of including a feature, i.e., the benefit Answers: What is the advantage of incorporating the user story in Agile? What problem are you solving?

User story ID: US01
User Role: any user
Description: As the user, I want to go to the landing page of the system

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Acceptance Criteria:
•
Supporting Information/Reference:
Output:
Dependencies: N/A
Integration: N/A

3.1.5 Use case diagram

Use Case ID:			
Use Case Name:			
Created By:		Last Updated By:	
Date Created:		Date Last Updated:	

Actors:	
Description:	
Preconditions:	
Postconditions:	
Normal Course:	
Alternative Courses:	
Exceptions:	
Includes:	
Priority:	
Frequency of Use:	
Business Rules	
Special Requirements:	
Assumptions:	
Notes and Issues:	

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

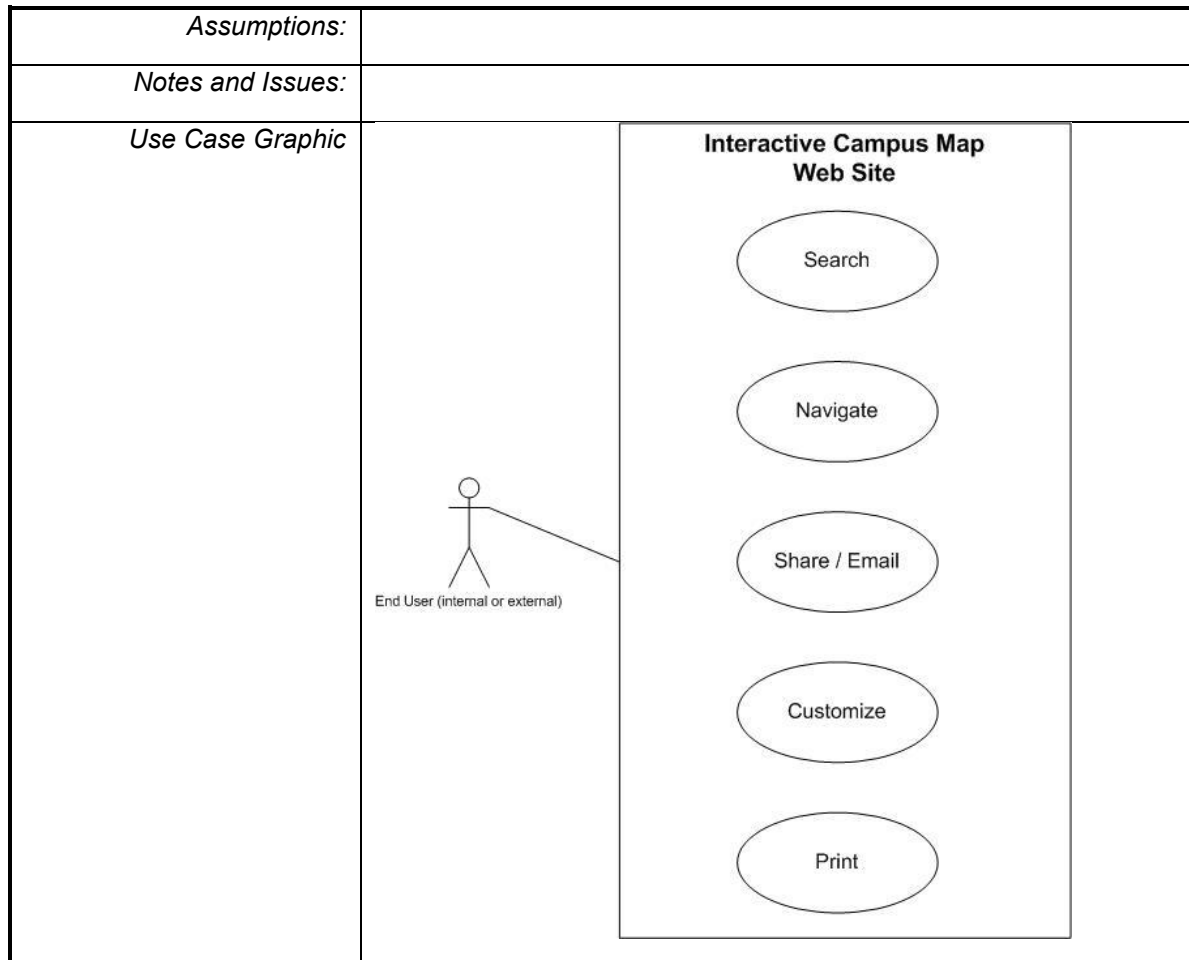
Use Case Graphic

Example of a completed use case:

Use Case ID:	1		
Use Case Name:	View Interactive Campus Map		
Created By:	Dan Sward	Last Updated By:	
Date Created:	4/19/09	Date Last Updated:	

Actors:	User
Description:	This use case describes the main way this interactive campus map will be used – as a web browser accessed application. The user accesses the appropriate URL and interacts with the functionality made available.
Preconditions:	Web browser opened, and interactive campus map URL accessed.
Postconditions:	User navigates from interactive campus map web site.
Normal Course:	1. Open browser 2. Navigate to campus map URL 3. Interact with the campus map using available functionality
Alternative Courses:	None
Exceptions:	None
Includes:	
Priority:	High
Frequency of Use:	Once per visit.
Business Rules	TBD...
Special Requirements:	• 24/7 access • Response times comparable to common web mapping solutions (e.g. Google Maps) • U of M accessibility requirements • U of M eCommunications requirements

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	



<Instructions for completing the Use Case Narrative are included here. Remove these instructions from the completed Business Requirements Document>.

Use Case Field Name	Definition
Use Case ID	Give each use case a unique numeric identifier, in hierarchical form: X.Y. Related use cases can be grouped in the hierarchy. Functional requirements can be traced back to a labeled Use Case.
Use Case Name	State a concise, results-oriented name for the use case. These reflect the tasks the user needs to be able to accomplish using the system. Include an action verb and a noun. Some examples: • View part number information.

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Use Case Field Name	Definition
	- Manually mark hypertext source and establish link to target. - Place an order for a CD with the updated software version
Created By	Include the name of the person who initially documented this Use Case.
Date Created	Enter the date on which the use case was initially documented
Date Last Updated	Enter the date on which the use case was most recently updated
Last Updated By	Include the name of the person who performed the most recent update to the use case description.
Actor	Enter the person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this Use Case.
Description	Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the Use Case.
Preconditions	List any activities that must take place, or any conditions that must be true, before the Use Case can be started. Number each precondition. Examples: - User's identity has been authenticated. - User's computer has sufficient free memory available to launch task
Post conditions	Describe the state of the system at the conclusion of the use case execution. Number each post condition. Examples: - Document contains only valid SGML tags. - Price of item in database has been updated with new value

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Use Case Field Name	Definition
Normal Course	Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description. This description may be written as an answer to the hypothetical question, "How do I <accomplish the task stated in the use case name>?" This is best done as a numbered list of actions performed by the actor, alternating with responses provided by the system.
Alternative Courses	Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place. Number each alternative course using the Use Case ID as a prefix, followed by "AC" to indicate "Alternative Course". Example: X.Y.AC.1
Exceptions	Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use case execution fails for some unanticipated reason. Number each exception using the Use Case ID as a prefix, followed by "EX" to indicate "Exception". Example: X.Y.EX.1
Includes	List any other use cases that are included ("called") by this use case. Common functionality that appears in multiple use cases can be split out into a separate use case that is included by the ones that need that common functionality.
Priority	Indicate the relative priority of implementing the functionality required to allow this use case to be executed. The priority scheme used must be the same as that used in the software requirements specification.

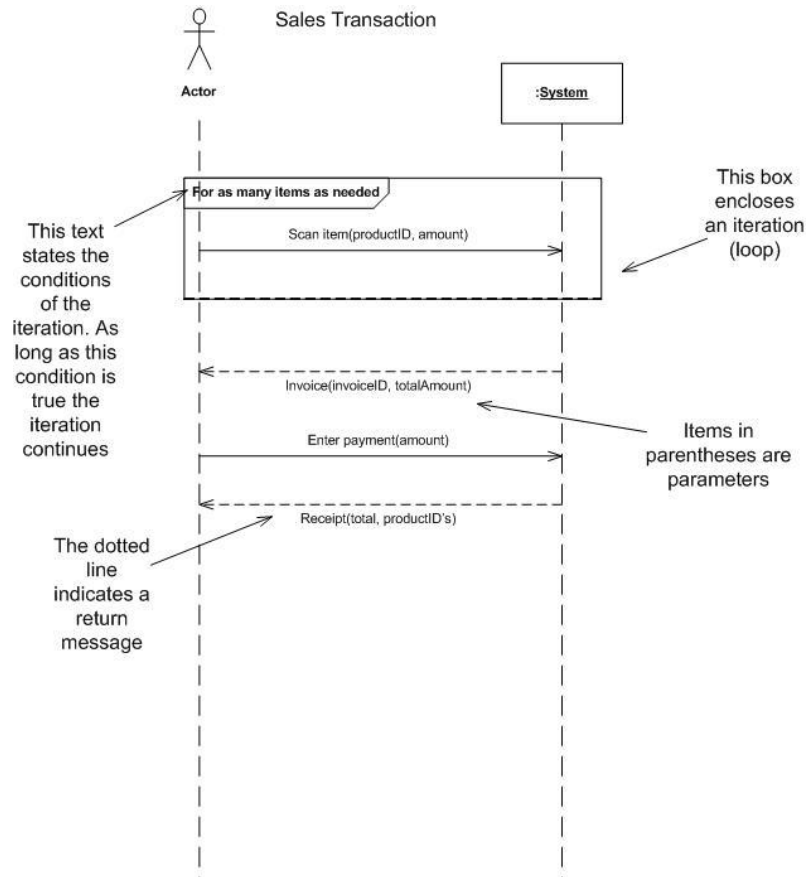
	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

Use Case Field Name	Definition
Frequency of Use	Estimate the number of times this Use Case will be performed by the actors per some appropriate unit of time.
Business Rules	List any business rules that influence this Use Case.
Special Requirements	Identify any additional requirements, such as nonfunctional requirements, for the use case that may need to be addressed during design or implementation. These may include performance requirements or other quality attributes.
Assumptions	List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.
Notes and Issues	List any additional comments about this use case or any remaining open issues or TBDs (To Be Determined) that must be resolved. Identify who will resolve each issue, the due date, and what the resolution ultimately is.

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

3.1.6 Sequence Diagram

Draw the sequence diagram for the relevant system feature or use case

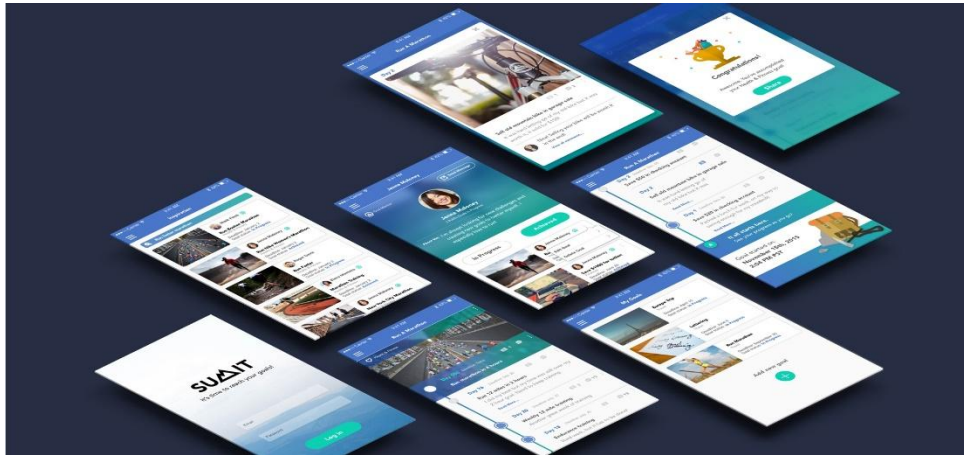


Sample

3.1.7 System Mock-up Screens

Draw the mock up screens for the project function or wire diagrams

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	



3.2 Functionality 2 (and so on)

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

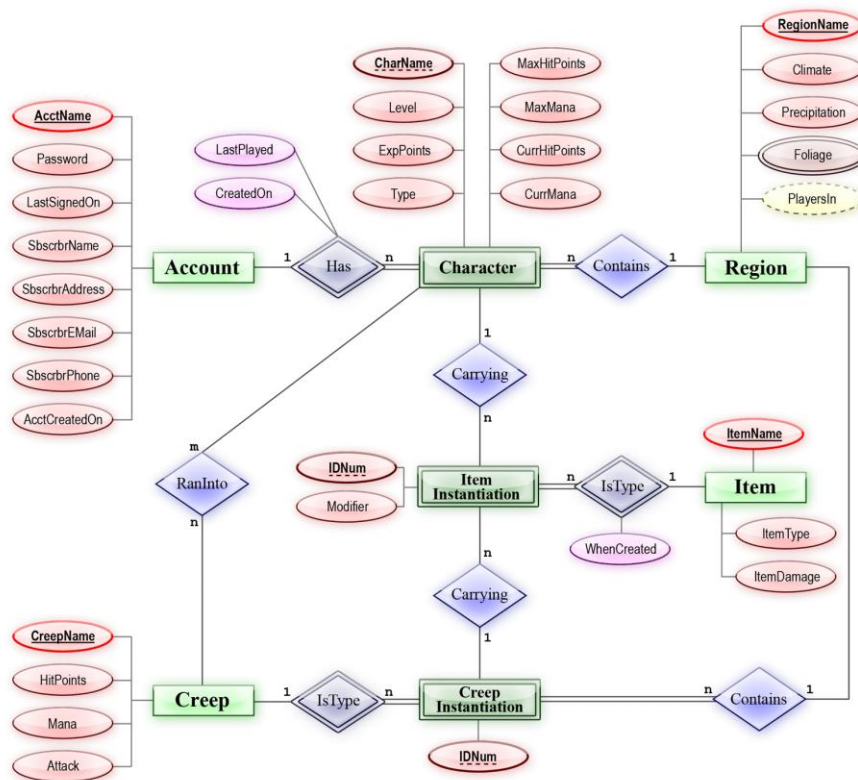
3.3 Logical Database Requirement

This section specifies the logical requirements for any information that is to be placed into a database. This may include:

- Types of information used by various functions
- Frequency of use
- Accessing capabilities
- Data entities and their relationships
- Integrity constraints
- Data retention requirements

If the customer provided you with data models, those can be presented here. ER diagrams (or static class diagrams) can be useful here to show complex data relationships.

Sample



	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

4.2 Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

4.3 Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

4.4 Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

ID	Characteristic	H/M/L	1	2	3	4	5	6	7	8	9	10	11	12
1	Correctness													
2	Efficiency													
3	Flexibility													
4	Integrity/Security													
5	Interoperability													
6	Maintainability													
7	Portability													
8	Reliability													

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

9	Reusability													
10	Testability													
11	Usability													
12	Availability													

Definitions of the quality characteristics not defined in the paragraphs above follow.

- *Correctness - extent to which program satisfies specifications, fulfills user's mission objectives*
- *Efficiency - amount of computing resources and code required to perform function*
- *Flexibility - effort needed to modify operational program*
- *Interoperability - effort needed to couple one system with another*
- *Reliability - extent to which program performs with required precision*
- *Reusability - extent to which it can be reused in another application*
- *Testability - effort needed to test to ensure performs as intended*
- *Usability - effort required to learn, operate, prepare input, and interpret output*

4.5 Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

OTHER REQUIREMENTS

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

5.1 Online User Documentation and Help System Requirements

[Describes the requirements, if any, for on-line user documentation, help systems, help about notices, etc.]

5.2 Purchased Components

[This section describes any purchased components to be used with the system, any applicable licensing or usage restrictions, and any associated compatibility/interoperability or interface standards.]

5.3 Interfaces

[This section defines the interfaces that must be supported by the application. It should contain adequate specificity, protocols, ports and logical addresses, etc, so that the software can be developed and verified against the interface requirements.]

5.3.1 User Interfaces

<Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.>

5.3.2 Hardware Interfaces

<Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.>

5.3.3 Software Interfaces

<Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Refer to documents that describe detailed application programming interface protocols. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

5.3.4 Communications Interfaces

<Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.>

5.4 Licensing Requirements

[Defines any licensing enforcement requirements or other usage restriction requirements that are to be exhibited by the software.]

5.5 Legal, Copyright and Other Notices

[This section describes any necessary legal disclaimers, warranties, copyright notices, patent notice, word mark, trademark, or logo compliance issues for the software.]

5.6 Applicable Standards

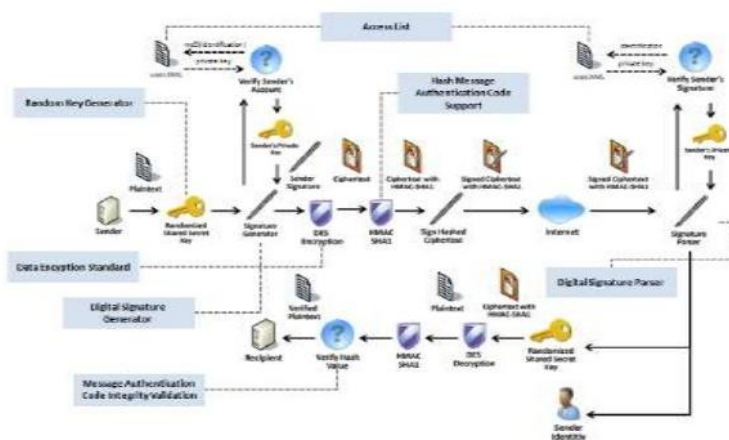
[This section describes by reference any applicable standards, (and the specific sections of any such standards that apply to the system being described). For example, this could include legal, quality and regulatory standards, industry standards for usability, interoperability, internationalization, operating system compliance, etc.]

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

SYSTEM ARCHITECTURE

[This section contains description of the system architecture. The system architecture should contain a description of the logical organization along with the reasoning behind the overall organization.]

System Architecture Example



6.1 Functional Architecture

[Functional architecture is a view of the system based on functional statements related to system operation. System functions are grouped into like kinds, providing a functional hierarchy structuring the interaction of system functionality. This area should contain a functional architecture diagram along with a description of each of the functional components.]

6.2 Logical Architecture

[Logical architecture is a view of the system based on logical abstractions derived from system functionality. These abstractions typically manifest as models showing major packaging of these logical constructs (package models) as well as class models showing low level logical constructs and relationships..]

6.3 Component Architecture

[This section contains description of the component architecture. A component is an abstraction related to eventual deployment of a part of the overall system. Components will have classes and interfaces mapped to them and manifest in the "real-world" as an executable or library (.dll, shared object library, static). Component architecture shows the dependencies between components and related interfaces.]

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

6.4 Physical Architecture

[This section contains description of the physical architecture The physical architecture depicts physical devices and how developed components map to those physical devices in a typical end user installation.]

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

APPENDIX A: GLOSSARY

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

APPENDIX B: ANALYSIS MODELS

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

	Version: <1.0>
Software Requirement Specification	Date: <dd-mmm-yyyy>
<document identifier>	

APPENDIX C: TO BE DETERMINED LIST

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>